



UNIVERSITA' DEGLI STUDI DI CAGLIARI

FACOLTÀ DI SCIENZE

Corso di Laurea Magistrale in Informatica

Data Mining: Report Finale

Gruppo G3.3

Componenti:

Aresu Matteo

60/99/00014

Mantega Gabriele

60/99/00006

Masala Gabriele

60/99/00004

ANNO ACCADEMICO 2025-2026

Indice

1	Introduzione	3
2	Dataset	4
3	Preprocessing	7
4	Applicazione Modelli	9
5	Discussione	11
6	Conclusioni	11

1 Introduzione

Questo report affronta il problema della classificazione dei documenti testuali, applicato all'analisi delle interazioni interne dei newsgroup, spazi online storicamente strutturati su infrastrutture decentralizzate, e dedicati al dibattito su topic ben determinati. L'obiettivo centrale del progetto è l'utilizzo di diversi algoritmi di classificazione, spaziando dai modelli storici tradizionali (shallow learning) ai modelli a combinazione (ensemble learning), al fine di predire il topic di un post partendo dal contenuto dei suoi messaggi. Lo studio non si è limitato all'analisi delle prestazioni dei classificatori, ma si estende alla comparazione dei metodi di rappresentazione geometrica dei dati in input. A questo scopo sono stati utilizzati due metodi di vettorizzazione: **TF-IDF** (Term Frequency-Inverse Document Frequency) e **GloVe** (Global Vectors for Word Representation). Il progetto è stato realizzato con l'ausilio dell'infrastruttura di Jupyter, il linguaggio di programmazione Python e delle sue librerie fondamentali, come pandas, scikit-learn, NumPy, Matplotlib.

Il presente report è organizzato nei seguenti passi:

- **Sezione 2 - Dataset** Analisi approfondita del dataset 20newsgroups, della sua struttura e delle sue categorie tematiche con approfondimento su quelle scelte.
- **Sezione 3 - Preprocessing** Descrizione del flusso di elaborazione e trasformazione dei dati, differenziando i due metodi di vettorizzazione scelti, con relativo approfondimento sulle scelte di implementazione prese durante lo sviluppo del progetto.
- **Sezione 4 - Applicazione modelli** Elenco dei classificatori utilizzati, con i relativi risultati di training e testing sul dataset e scelte implementative effettuate sugli iperparametri.
- **Sezione 5 - Discussione** Argomentazione esaustiva sui risultati ottenuti con considerazioni argomentate e rappresentazione visiva dei dati, mediante tabelle e grafici informativi.

2 Dataset

Il dataset di riferimento utilizzato per la sperimentazione è **20newsgroups**, un corpus di circa 18.000 pubblicazioni ampiamente utilizzata nel Natural Language Processing (NLP), e distribuite all'interno di venti differenti categorie tematiche. Una caratteristica di questo dataset è l'organizzazione in forma gerarchica dei suoi topic, formalizzata dalla seguente struttura: `macrocategoria.sottocategoria` (con la possibilità di inserire ulteriori sottocategorie, successivamente alla prima). La documentazione relativa e i dati di partenza sono consultabili liberamente al seguente link: <https://www.kaggle.com/datasets/crawford/20-newsgroups>.

Gruppo di Discussione	Descrizione
<code>rec.autos</code>	Discussioni su motori, recensioni, manutenzione e novità automobilistiche.
<code>rec.sport.baseball</code>	Spazio per appassionati di baseball: analisi, statistiche e campionati.
<code>sci.electronics</code>	Forum su progettazione di circuiti, componenti e teoria elettronica.
<code>sci.med</code>	Discussioni su medicina, salute, ricerche scientifiche e trattamenti.
<code>sci.space</code>	Spazio dedicato all'esplorazione spaziale, astronomia e tecnologia dei razzi.

Tabella 2.1: Elenco dei newsgroup utilizzati nel progetto e relative descrizioni

Ai fini dell'elaborazione del dataset, è stato utilizzato solo un sottoinsieme di cinque categorie per analizzare il comportamento dei classificatori, di cui alcune rappresentanti tematiche potenzialmente sovrapponibili concettualmente. La totalità delle categorie utilizzate in questo scopo sono riportate in Tabella 2.1. Dal punto di vista strutturale, ogni singolo documento del dataset corrisponde a un post creato da un utente. In linea con gli standard Usenet, ogni post è strutturato nel seguente modo:

- **Header:** una sezione iniziale del documento, dove vengono inseriti alcuni metadati tecnici, come l'indirizzo email del mittente, il server di provenienza e l'oggetto del messaggio.
- **Body:** il testo effettivo del post scritto dall'utente, un insieme di frasi in linguaggio naturale costituenti il vero nucleo informativo.
- **Footer:** una sezione finale dedicata alle firme automatiche o citazioni personali.

Un post di esempio, che rispetta questa struttura, è osservabile in Figura 2.1.

Il dataset è stato caricato in locale tramite la funzione `datasets.fetch_20newsgroups()` di scikit-learn ed è stato successivamente ripartito in **training set** e **test set**, come rappresentato in Figura 2.2. In Figura 2.3 viene riportata la distribuzione delle classi all'interno di training e test set, tenendo presente che queste rappresentano i gruppi di discussione. Si noti che le classi sono distribuite in maniera omogenea all'interno del dataset. Data l'assenza di sbilanciamenti nel dataset siamo in grado di comparare più equamente le prestazioni tra le diverse classi.

From: baalke@kelvin.jpl.nasa.gov (Ron Baalke)
Subject: Galileo Update - 04/15/93
Organization: Jet Propulsion Laboratory
Lines: 113
Distribution: world
NNTP-Posting-Host: kelvin.jpl.nasa.gov
Keywords: Galileo, JPL
News-Software: VAX/VMS VNEWS 1.41

Forwarded from Neal Ausman, Galileo Mission Director

GALILEO
MISSION DIRECTOR STATUS REPORT
POST-LAUNCH
April 9 - 15, 1993

SPACECRAFT

1. On April 9, real-time commands were sent, as planned, to reacquire celestial reference completion of the Low Gain Antenna (LGA-2) swing/Dual Drive Actuator (DDA) hammer activities.
...

Figura 2.1: Estratto di un post del topic sci.space

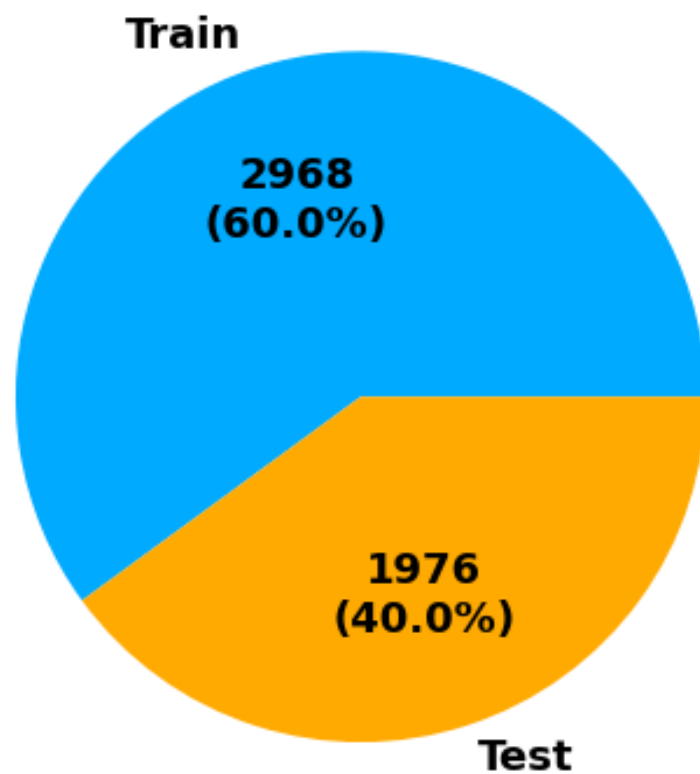


Figura 2.2: Distribuzione degli oggetti tra training set e test set

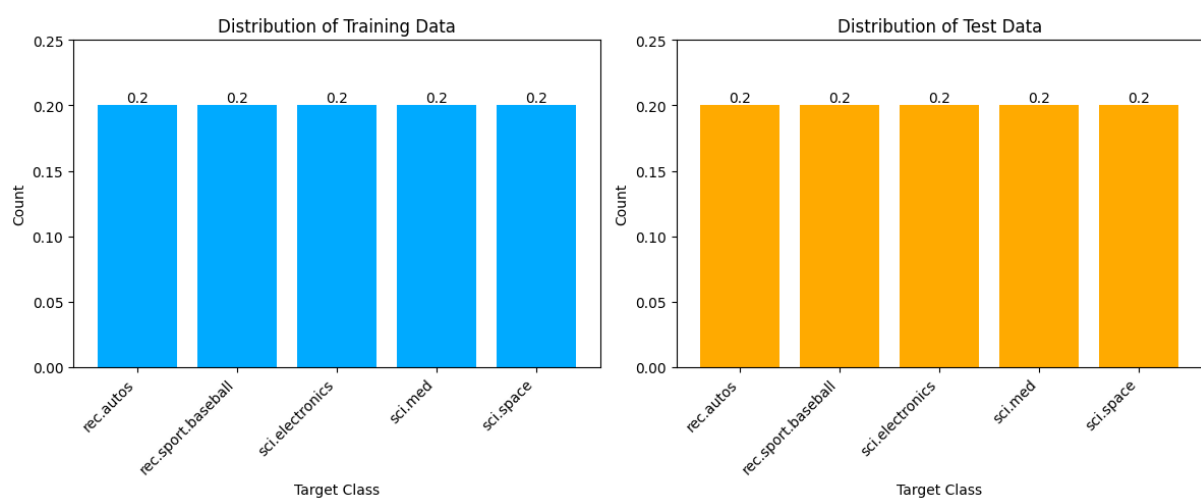


Figura 2.3: Distribuzione delle classi nel dataset in esame

3 Preprocessing

La fase di preprocessing è finalizzata alla conversione dei dati testuali grezzi, contenuti nei documenti del dataset, in rappresentazioni numeriche strutturate, tramite un processo chiamato vettorizzazione. In questo progetto sono stati implementati e confrontati due approcci metodologici: la rappresentazione statico-frequenziale **TF-IDF** e l'utilizzo di vettori semantici densi tramite **embeddings GloVe**. Entrambe le metodologie sono state configurate per lavorare su uno spazio lineare a cento dimensioni, come da specifiche.

- **Vettorizzazione TF-IDF**

Il metodo TF-IDF (Term Frequency-Inverse Document Frequency) costruisce un dizionario interno al modello, inserendo al suo interno ogni parola presente nel dataset. Per ogni parola presente nel dizionario, viene calcolato il suo punteggio IDF. Il nucleo dell'algoritmo prevede il calcolo della frequenza di una parola all'interno del singolo post (frequenza intra-documento) pesata inversamente rispetto alla sua diffusione nell'intero dataset (frequenza inter-documento). L'intuizione è quella di restituire un valore TF-IDF più alto alle parole rare, in quanto in possesso di elevato potere discriminante per l'identificazione del topic di appartenenza dell'intero messaggio. In Equazione (1) è riportata la formula utilizzata da `TfidfVectorizer()`, implementato da scikit-learn, per il calcolo del punteggio TF-IDF per un dato termine t all'interno di un documento d . Si noti che i parametri di *default* prevedono lo *smoothing* aggiungendo 1 per evitare la divisione per zero.

$$\text{TF-IDF} = \text{tf}(t, d) \times \text{idf}(t)$$
$$\text{idf}(t) = \log \left(\frac{1 + N}{1 + \text{df}(t)} \right) + 1 \quad (1)$$

- **Embedding Semantici GloVe**

A differenza dell'approccio probabilistico e lessicale di TF-IDF, GloVe si appoggia ad un dizionario di vettori generato da un modello pre-addestrato. Ogni parola viene proiettata su uno spazio geometrico continuo a N dimensioni, in cui le coordinate precise di un termine sono in grado di rappresentare il suo significato semantico. Questa rappresentazione ha il vantaggio che, parole con significato semantico simile, risulteranno vicine geometricamente tramite distanza euclidea. Ciò permette ai classificatori di intercettare relazioni tra parole sinonime e contesti logici che il semplice calcolo delle frequenze non sarebbe in grado di scoprire.

In entrambi i casi, prima della vettorizzazione, viene eseguito un passo di normalizzazione del testo e di pulizia dalle stop-words. Queste sono parole di uso comune in un linguaggio che non trasmettono un significato semantico saliente, che tuttavia fanno parte della grammatica. Un esempio di tali parole sono gli articoli, presenti sia nella lingua italiana che in quella inglese. Nel caso di TF-IDF, questa fase di pulizia avviene internamente alla funzione `TfidfVectorizer()` impostando il parametro `stop_words="english"`. Nel caso di GloVe, abbiamo implementato le seguenti funzioni:

- `preprocess_text(text)`: esegue il lowercasing, elimina caratteri speciali e punteggiatura, rimuove le stop-words (ovvero le occorrenza di `sktext.ENGLISH_STOP_WORDS` all'interno del documento) ed effettua la tokenization delle parole;
- `load_embeddings(file_path)`: carica il dizionario degli embeddings a partire dal file `glove.6B.100d.txt`, che presenta al suo interno un vettore di 100 valori numerici per ciascuna parola;
- `get_document_embedding(tokens, glove_embeddings)`: per ciascun token di un documento, ottiene l'embedding corrispondente, se presente nel dizionario GloVe, e calcola la media di tutti questi embedding per ottenere un unico vettore che rappresenta il documento. Se nessun token è presente all'interno del dizionario, il document embedding corrispondente è il vettore nullo.

4 Applicazione Modelli

La fase sperimentale ha previsto l'addestramento di sei modelli di classificazione multiclasse. Di seguito si propone, per ciascun algoritmo, una breve descrizione unita alla giustificazione metodologica della scelta dei relativi iperparametri.

- **Decision Tree (DT)**

Classificatore di shallow learning, basato su un albero decisionale in cui i nodi foglia rappresentano le classi e i nodi genitore rappresentano criteri decisionali. Nello specifico, la sua implementazione su scikit-learn avviene tramite albero binario. È stata impostata una profondità massima `max_depth=8`, che è piuttosto limitata, per evitare overfitting sul dataset. Una misura di profondità eccessivamente alta potrebbe infatti condurre l'albero a imparare i criteri esatti per la classificazione. Invece, in tale modo abbiamo al più $2^{(8-1)} = 128$ nodi decisionali, che si avvicinano al numero di features utilizzate. È stato impostato un limite inferiore al numero di campioni necessari in una classe per eseguire uno split. Il valore scelto è `min_samples_split=0.01`, corrispondente all'1% del dataset. Ciò garantisce una crescita più controllata dell'albero.

- **K-Nearest Neighbors (KNN)**

Ampiamente considerato uno dei più semplici algoritmi di machine learning. Secondo il KNN, un oggetto è classificato in base alla maggioranza dei voti dei suoi k vicini (con k un iperparametro intero positivo). La sua implementazione su scikit-learn non ha una vera fase di addestramento, si limita a inserire l'intero dataset sulla memoria centrale. Questa implementazione ha il vantaggio di avere un costo computazionale pari a $O(1)$, tuttavia risulta in pessime prestazioni con dataset molto grandi. La scelta degli iperparametri è stata guidata dalla natura intrinseca dei dati testuali vettorializzati. In primo luogo è stata adottata la metrica di similarità del coseno (`metric='cosine'`), alternativamente alla più classica distanza euclidea. Questa scelta deriva dal fatto che, usando rappresentazioni geometriche vettoriali (GloVe e TF-IDF), l'orientamento dei vettori cattura il significato semantico e concettuale in modo più robusto rispetto alla loro norma geometrica. Parallelamente, il numero di vicini è stato fissato a quindici (`n_neighbors=15`), per avere un bilanciamento ottimale tra bias e varianza. Un valore di K eccessivamente alto o eccessivamente basso potrebbe portare il modello a condizioni di, rispettivamente, overfitting o underfitting.

- **Multi Layer Perceptron (MLP)**

Modello di rete neurale artificiale, costruita con strati multipli di nodi in un grafo diretto, di cui ogni strato è direttamente connesso al successivo. Il suo obiettivo è mappare un insieme di dati di ingresso in un insieme di dati di uscita appropriati. L'architettura della rete neurale artificiale è stata modellata con una scelta degli iperparametri in grado di catturare le relazioni non lineari dei vettori testuali. Il numero di nodi di input scelto è pari a 64, ovvero la potenza di 2 più alta e inferiore di 100. Così facendo la rete viene forzata a comprimere le informazioni essenziali e ad scartare quelle poco salienti. Il numero di nodi dell'hidden layer contrae la dimensionalità a 32 neuroni, agendo come un ulteriore imbuto. Gli iperparametri scelti per l'input layer e l'hidden layer sono `hidden_layer_sizes=(64, 32)`. Per quanto riguarda il processo di ottimizzazione, il limite massimo delle epoche è stato elevato a trecento, per garantire un margine computazionale sufficiente a convergere verso un minimo locale stabile (`max_iter=300`). Al fine di prevenire fenomeni di overfitting, è stata attivata la regolarizzazione dinamica tramite `early_stopping=True`.

- **Random Forest (RF)**

Classificatore di ensemble learning, ottenuto dall'aggregazione tramite bagging di diversi Decision Tree, nasce come soluzione atta a ridurre l'overfitting del training set di quest'ultimi. Gli iperparametri scelti per gli estimatori sono stati quelli di default, poichè le loro prestazioni, lavorando parallelamente e con output congiunto, sono soddisfacenti. Inoltre, poichè il RF permette ai suoi estimatori di lavorare in parallelo e non sequenzialmente, abbiamo impostato come numero di quest'ultimi pari a 200, un numero più alto della media, ma comunque non sufficiente per l'overfitting (`n_estimators=200`).

- **Adaptive Boosting (AdaBoost, AB)**

L'intuizione che sta alla base dell'Adaboost risiede nell'addestramento sequenziale di un insieme di classificatori deboli (weak learners). A differenza dei metodi di bagging (come, ad esempio, il RF), in cui i modelli operano in parallelo e con parità di potere decisionale, Adaboost adotta un approccio in cui ogni nuovo stimatore viene addestrato focalizzandosi prevalentemente sui punti classificati erroneamente dagli scorsi stimatori.

- **eXtreme Gradient Boosting (XGBoost, XGB)**

Evoluzione dell'AdaBoost, basato sul paradigma del Gradient Tree Boosting.

5 Discussione

6 Conclusioni